



Accessing x86-Specific I/O-Mapped Hardware in Linux

This article, which is part of the series on Linux device drivers, continues the discussion on accessing hardware in Linux.

The second day in the Linux device drivers' laboratory was expected to be quite different from the typical software-oriented class. Apart from accessing and programming architecture-specific I/O mapped hardware in x86, it had a lot to offer first-timers with regard to reading hardware device manuals (commonly called data sheets) and how to understand them to write device drivers. In contrast, the previous session about generic architecture-transparent hardware interfacing was about mapping and accessing memory-mapped devices in Linux without any device-specific details.

x86-specific hardware interfacing

Unlike most other architectures, x86 has an additional hardware accessing mechanism, through direct I/O mapping. It is a direct 16-bit addressing scheme, and doesn't need mapping to a virtual address for access. These addresses are referred to as port addresses, or ports. Since this is an additional access mechanism, it has an additional set of x86 (assembly/machine code) instructions. And yes, there are the input instructions *inb*, *inw*, and *inl* for reading an 8-bit byte, a 16-bit word, and a 32-bit long word, respectively, from I/O mapped devices, through ports. The corresponding output instructions are *outb*, *outw*

and *outl*, respectively. The equivalent C functions/macros (available through the header `<asm/io.h>`) are as follows:

```
u8 inb(unsigned long port);
u16 inw(unsigned long port);
u32 inl(unsigned long port);
void outb(u8 value, unsigned long port);
void outw(u16 value, unsigned long port);
void outl(u32 value, unsigned long port);
```

The basic question that may arise relates to which devices are I/O mapped and what the port addresses of these devices are. The answer is pretty simple. As per x86-standard, all these devices and their mappings are predefined. Figure 1 shows a snippet of these mappings through the kernel window `/proc/ioports`. The listing includes predefined DMA, the timer and RTC, apart from serial, parallel and PCI bus interfaces, to name a few.

Simplest: serial port on x86

For example, the first serial port is always I/O mapped from 0x3F8 to 0x3FF. But what does this mapping mean? What do we do with this? How does it help us to use the serial port? That is where a data-sheet of the corresponding